

TECH FORCE: GALACTIC
ARCHITECTURE

LINGUAGEM ASSEMBLY X86



7

LISTA DE FIGURAS

Figura 1 - Exemplo da linguagem Assembly	6
Figura 2 - ReactOS rodando o emulador PicoDrive	8
Figura 3 - Exemplo de compilação usando GCC vs Movfuscator	10
Figura 4 - Adição em Assembly	16
Figura 5 - Subtração em Assembly	16
Figura 6 - Incrementando valor por 1 em Assembly	17
Figura 7 - Decrementando valor por 1 em Assembly	18
Figura 8 - Divisão em Assembly	19
Figura 9 - Multiplicação em Assembly	19
Figura 10 - Exemplo da <i>Stack</i>	21
Figura 11 - Prólogo da função <i>main</i>	23
Figura 12 - Epílogo da função <i>main</i>	23
Figura 13 - Inicializando as variáveis (A, B e <i>result</i>)	23
Figura 14 - Instruções e chamadas de função para receber o <i>input</i> do usuário	24
Figura 15 - Exemplo de <i>input</i> do usuário no cmd	24
Figura 15 - Parâmetros sendo jogados na <i>Stack</i> e chamada para nossa função <i>check()</i>	24
Figura 17 - Função <i>check()</i>	26
Figura 18 - Primeiro bloco	26
Figura 19 - Segundo bloco	27
Figura 20 - Terceiro bloco	27
Figura 21 - Exemplo de convecção de chamada <i>__cdecl</i>	29
Figura 22 - Exemplo de convecção de chamada <i>__stdcall</i>	29
Figura 23 - Exemplo de convecção de chamada <i>__fastcall</i>	30
Figura 24 - Laço de repetição FOR na linguagem assembly	32
Figura 25 - Laço de repetição WHILE na linguagem assembly	34
Figura 26 - Laço de repetição DO WHILE na linguagem assembly	35
Figura 27 - IDA <i>Free disassembler</i>	36
Figura 28 - Exemplo de decompilação utilizando o IDAFree	37
Figura 29 - Instalando o Ghidra	38
Figura 30 - Instalando o JDK	38
Figura 31 - Configuração do sistema	39
Figura 32 - Variáveis de ambientes do Windows	39
Figura 33 - Adicionando o path para os binários do JDK	40
Figura 34 - Executando o Ghidra	40
Figura 35 - Ghidra Instalado	41
Figura 36 - <i>Overview</i> do binário a ser analisado usando o Ghidra	41
Figura 37 - CodeBrowser	41
Figura 38 - Ghidra	42
Figura 39 - x64dbg em sua versão 32-bit	43
Figura 40 - <i>Loader</i> do sistema Windows	43
Figura 41 - <i>EntryPoint</i> do programa a ser debugado	43
Figura 42 - <i>Web disassembler</i> Godbolt	45
Figura 43 - <i>Web disassembler</i> Dogbolt	45

LISTA DE TABELAS

Tabela 1 – Exemplos de instruções MOV.....	9
Tabela 2 – Formato das instruções	11
Tabela 3 – Instruções e <i>Opcodes</i>	11
Tabela 4 – Registradores da arquitetura x86.....	13
Tabela 5 – Instruções de adição, subtração e incrementação.....	15
Tabela 6 – Instruções para multiplicação e divisão	18
Tabela 7 – Exemplos comuns de instruções aritméticas lógicas e <i>Shifiting</i>	20
Tabela 8 – Instruções da <i>Stack</i>	21
Tabela 9 – Comandos básicos do x64dbg.....	44

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de validação do membro BeingDebugged	14
Código-fonte 2 – Exemplo de cálculo em C	15
Código-fonte 3 – Operação de soma em Assembly	16
Código-fonte 4 – Incrementando valor por 1	17
Código-fonte 5 – Exemplo de divisão e multiplicação em C.....	18
Código-fonte 6 – Exemplo de função	22
Código-fonte 7 – Exemplo de uso do laço de repetição FOR	31
Código-fonte 8 – Exemplo de uso do laço de repetição FOR	33
Código-fonte 9 – Exemplo de uso do laço de repetição DO WHILE	34

SUMÁRIO

LINGUAGEM ASSEMBLY X86.....	6
1 INTRODUÇÃO	6
2 PARA QUE APRENDER ASSEMBLY?.....	7
3 ASSEMBLY 101	10
3.1 Recapitulando os registradores.....	12
3.1.1 General Registers	13
3.1.2 Segment Registers.....	14
3.1.3 EFLAGS Registers	14
3.1.4 EIP Register	15
3.2 Arithmetic Instructions	15
3.3 Pilha (Stack).....	20
3.4 Function Calls.....	22
3.5 Conditionals & Branching	25
3.6 Call Conventions	28
3.6.1 Cdecl	28
3.6.2 Stdcall	29
3.6.3 Fastcall	30
3.7 Loops	30
3.7.1 FOR loops	31
3.7.2 While & Do While	33
3.7.2.1 While	33
3.7.2.2 Do While.....	34
4 FERRAMENTAS	35
4.1 IDA Pro.....	35
4.2 Ghidra	37
4.3 X64dbg.....	42
5 DICAS EXTRAS	44
5.1 Godbolt & Dogbolt.....	45
5.2 Livros e Dicas:.....	45
CONCLUSÃO	46
REFERÊNCIAS.....	47

LINGUAGEM ASSEMBLY X86

1 INTRODUÇÃO

Se eu te falar que todo software do mundo é *open source* você acreditaria? Você deve achar que eu estou louco, mas não! Para você encarar o mundo moderno como uma biblioteca infinita de diferentes códigos você precisa aprender a linguagem Assembly.

Essa é uma linguagem de baixo nível, mas não é muito diferente de linguagens como Python, ou C/C++. Porém muitas pessoas têm medo quando veem algo desse tipo:

```
.text:00401040 _main      proc near          ; CODE XREF: sub_401149+F5↓p
.text:00401040                                     = dword ptr 4
.text:00401040 argv       = dword ptr 8
.text:00401040 envp       = dword ptr 0Ch
.text:00401040
.v .text:00401040      push     esi
.text:00401041      xor      esi, esi
.text:00401043      loc_401043:      ; CODE XREF: _main+2A↓j
.text:00401043      cmp      esi, 5
.text:00401046      jz       short loc_401057
.text:00401048      cmp      esi, 0Ah
.text:0040104B      jz       short loc_401057
.text:0040104D      mov      eax, offset Format ; "%d\n"
.text:00401052      cmp      esi, 0Fh
.text:00401055      jnz      short loc_40105C
.text:00401057      loc_401057:      ; CODE XREF: _main+6↑j
.text:00401057      ; _main+8↑j
.text:00401057      mov      eax, offset aHereIsTheNumbe ; "Here is the number: %d\n"
.text:0040105C      loc_40105C:      ; CODE XREF: _main+15↑j
.text:0040105C      push     esi
.text:0040105D      push     eax
.text:0040105E      call     sub_401010
.text:00401063      inc      esi
.text:00401064      add      esp, 8
.text:00401067      cmp      esi, 14h
.text:0040106A      jle      short loc_401043
.text:0040106C      call     ds:getchar
.text:00401072      xor      eax, eax
.text:00401074      pop      esi
.text:00401075      retn
.text:00401075 _main      endp
```

Figura 1 - Exemplo da linguagem Assembly
Fonte: Elaborada pelo autor (2023)

Mas eu posso te garantir que cada instrução presente na figura “Exemplo de linguagem Assembly” possui um papel importante para o funcionamento do nosso simples programa. Você apenas precisa encarar o Assembly como uma linguagem de alto nível como C.

Neste capítulo iremos aprender sobre a linguagem Assembly, o quão poderoso esse conhecimento pode lhe ajudar como um profissional de cibersegurança ou

desenvolvedor de software. Abrindo portas para o mundo de desenvolvimento de exploits, engenharia reversa e análise de malware. Esse conhecimento também vai te ajudar muito como desenvolvedor, pois uma vez que você consegue ler códigos que foram compilados você começa a criar uma visão do que pode ou não ser otimizado em seu código fonte. Dessa forma criando software ainda mais performáticos.

Mas para isso eu devo dizer, não será da noite para o dia, ou só com este capítulo que você irá virar um mestre da linguagem Assembly. Muito pelo contrário, esse é um conhecimento que exige **muito tempo de dedicação** para começar a interpretar cada instrução em seu contexto. Para isso existem diferentes métodos para aprender sobre Assembly.

Juntos iremos fazer um overview da linguagem Assembly contextualizando junto a construções de código como: condições IF/ELSE, Laços de repetição (while, for, do/while), etc...

Então se prepare para adquirir um conhecimento que irá mudar sua percepção sobre software.

2 PARA QUE APRENDER ASSEMBLY?

Antes de entrarmos a fundo no capítulo, vamos ver algumas curiosidades de como a linguagem Assembly é poderosa e muito legal de se aprender.

Quando eu disse que todo o software do mundo pode ser *open source* (código aberto), você realmente achou que eu estava mentindo? Então você achou errado. Você já ouvir falar do ReactOS? O ReactOS é uma versão *open source* do sistema operacional Windows, sim isso mesmo.

Esse projeto se deu origem por volta de 1996 tendo como objetivo criar um clone do Windows 95. Seu nome a princípio era FreeWin95, porém o nome ReactOS foi cunhado por conta da insatisfação do grupo por parte do monopólio da Microsoft na época. Um dos principais desenvolvedores desses projetos foram: Alex Ionescu, Alexey Ivanov, Hartmut Birr.

Boa parte desse projeto se dou origem da engenharia reversa do sistema Windows. É claro, existem diversos motivos de se isso e legal ou ilegal, *copyright* etc. Porém não vamos entrar em detalhes a respeito desse assunto. Mas para caso você tenha dúvidas, procure sobre DRM (ou *Digital Rights Management*).

Caso queira saber mais informações sobre *Digital Rights Management*, acesse: <https://rockcontent.com/br/blog/drm/>

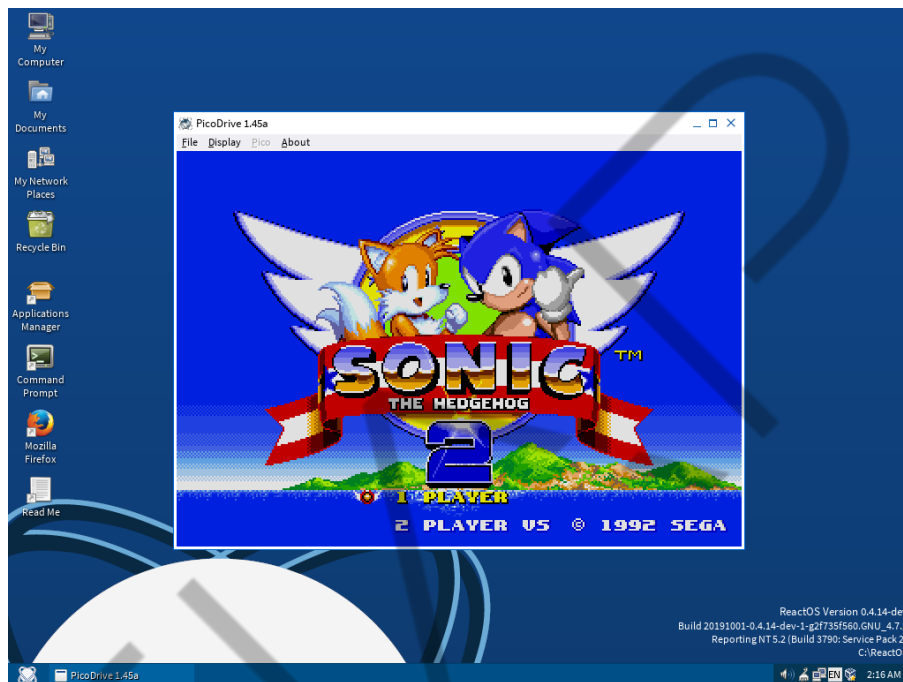


Figura 2 - ReactOS rodando o emulador PicoDrive

Fonte: REACTOS (s. d.)

Sim, sua aparência não é a das mais bonitas. Porém, o conhecimento sobre Windows Internals que se pode ser obtido com seu código fonte é de imenso valor. Ajudando no desenvolvimento de exploits, malware, e softwares para sistemas Windows. Por experiência própria, tente instalá-lo em uma VM (Virtual Machine), e tente executar algo que você normalmente usa no seu dia a dia, explore o sistema.

E se eu te perguntar qual é a única instituição necessária para se ter em um código, você saberia responder? Não? Então vamos **mover** para outro tópico.

Ok, não irei deixá-lo na dúvida! Pois com apenas a instituição **MOV** é possível fazer diversas operações. Essa instituição como o próprio nome sugere é usada para mover dados. Como podemos ver na tabela abaixo.

Instruções	Descrição
mov eax, ebx	Copia o valor de EBX para o registrador EAX.
mov eax, 0x42	Copia o valor 0x42 para o registrador EAX.
mov eax, [0x4037C4]	Copia 4 bytes do endereço de memória 0x4037C4 para o registrador EAX.
mov eax, [ebx]	Copia 4 bytes do endereço de memória no registrador EBX para o EAX.
mov eax, [ebx+esi*4]	Copia 4 bytes do endereço de memória do resultado da equação $ebx+esi*4$ para o registrador EAX.

Tabela 1 – Exemplos de instruções MOV

Fonte: Elaborada pelo autor (2023)

Como podemos ver a instrução **MOV** permite realizar operações de diferentes valores, veremos isso um pouco mais claro a seguir. Ok então para que foi tudo isso? Basicamente, quando você está analisando um malware, ou qualquer outro software comercial, normalmente haverá alguma forma de *trick*. **Tricks são ações performadas pelo malware para dificultar a análise.** Ou alguma forma de ofuscação de código **para ocultar certas informações.** A questão é até onde o engenheiro reverso gastaria tempo para tentar entender o comportamento de um software?

Foi aí que o Christopher Domas, um engenheiro reverso decidiu criar o MoVfuscator. Esse é um compilador para programas em C, que tem como objetivo transformar todas as instruções em simples operações **MOV**. Abaixo podemos ver um exemplo do mesmo programa compilado usando o GCC e o Movfuscator.

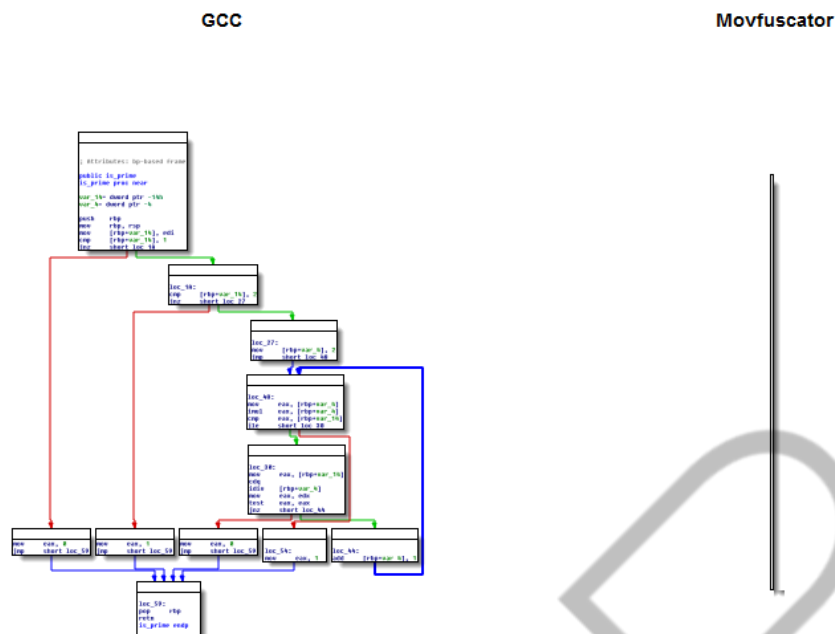


Figura 3 - Exemplo de compilação usando GCC vs Movfuscator
Fonte: Elaborada pelo autor (2023)

Qual é a utilidade disso no mundo real? Sinceramente, nem uma. Porém, essa foi a solução bem interessante que o Christopher desenvolveu com o intuito de que praticamente nem um engenheiro reverso sensato irá ficar olhando um único bloco de instruções **MOV**. Sua ideia aqui era fazer com que o analista desistisse de tentar entender o funcionamento do programa com base na análise estatística do código.

Bom agora que vimos algumas curiosidades sobre a linguagem Assembly. Porém, existem muitas outras, vale a pena fazer uma pesquisa sobre o assunto. Vamos agora nos aventurar nos detalhes a respeito dessa linguagem.

3 ASSEMBLY 101

Como vimos a linguagem Assembly é muito poderosa, então vamos começar pelo básico e ao final desse capítulo você já estará um pouco mais familiarizado com a linguagem. Aqui vale ressaltar que iremos trabalhar com a **notação IA-32**, por ela ser mais limpa e comumente abordada em livros. Porém, ter uma noção básica de outras notações não faz mal a ninguém.

A primeira coisa que precisamos saber é que as instruções em assembly x86 são formadas por *mnemonic* (ou mnemônico), zero ou mais *operands* (ou operadores). Na tabela “Formato das instruções” é possível visualizar essa representação. O mnemônico é uma palavra que identifica uma instrução a ser executada, como o **MOV**. Já os operadores são normalmente usados para identificar informações usadas pelas instruções, como registradores ou dados.

Mnemônico	Operador de Destino	Operador de Origem
mov	ecx	0x42

Tabela 2 – Formato das instruções
Fonte: Elaborada pelo autor (2023)

Cada instrução possui seu próprio *Opcodes* (código de operação), esse são representados em Hexadecimal, que falam para CPU qual operação o programa precisa performar. A ideia básica de um desmontador (*disassembler*) como o IDA, é traduzir esses opcodes para instruções que nós como seres humanos conseguimos ler de forma mais fácil.

Instrução	mov ecx, 0x42
Opcodes	B9 42 00 00 00

Tabela 3 – Instruções e *Opcodes*
Fonte: Elaborada pelo autor (2023)

Na Tabela “Instruções e *Opcodes*”, o valor 0xB9 corresponde a instrução **mov ecx**, e o valor **0x42000000** representa a o valor 0x42 em little-endian.

É bastante importante entender sobre a diferenças entre os *endianness* dos dados, pois ele descreve se o byte mais significativo (*big-endian*) ou menos significativo (*little-endian*) é ordenado primeiro (no menor endereço) dentro de um item de dados maior. Pense da seguinte forma, a troca de *endianness* é bastante comum durante a comunicação de rede.

Digamos que o endereço IP 127.0.0.1 representando no formato big-endian fique da seguinte forma na rede 0x7F000001, já quando esse endereço vai para a memória do programa ele é invertido para little-endian 0x0100007F. Novamente é muito importante lembrar desse detalhe.

Agora falando um pouco sobre os operadores. Esses são usados para identificar os dados usados pela instrução. Existem três tipos de operadores que podem ser usados.

1. **Immediate (imediato)** – Esses operadores são valores fixos como 0x42, mostrado na Tabela 2.
2. **Register (registrador)** – Esses operadores se referem aos registradores como o ecx, mostrado na Tabela 2.
3. **Memory Address (endereço de memória)** – Esses operadores se referem a endereços de memória que contêm algum valor a ser utilizado pelo programa. Normalmente representando como um valor fixo, registrador, ou equação entre colchetes, como [eax].

3.1 Recapitulando os registradores

Já vimos um conteúdo sobre registradores, mas vamos contextualizar eles de forma melhor junto ao Assembly e como visualizar sua utilização feita pelo código.

Como já sabemos os registradores são pequenos espaços de armazenamento temporário disponível na CPU, os dados armazenados neles permite um acesso de forma muito mais rápida. Vimos que existe diferentes tipos de registradores como:

- **General Registers (Registradores geral)** – São usados pela CPU durante a execução.
- **Segment registers (Registradores de segmentos)** – São usados para rastrear as sessões de memória.
- **Status Flags** – São usados para tomadas de decisões.
- **Instruction Pointers (Instruções do Ponteiro)** – São usados para manter o controle da próxima instrução a ser executada.

General Registers	Segment Registers	Status Register	Instruction Registers
EAX (AX, AH, AL)	CS	EFLAGS	EIP
EBX (BX, AH, AL)	SS		
ECX (CX, CH, CL)	DS		
EDX (DX, DH, DL)	ES		
EBP (BP)	FS		
ESP (SP)	GS		
ESI (SI)			

Tabela 4 – Registradores da arquitetura x86
 Fonte: Elaborada pelo autor (2023)

Todos os registradores de uso geral são de 32 bits e podem ser referenciado como 32 ou 16 bits em código assembly. Por exemplo, EDX é usado para referenciar um registrador de 32-bit, já o DX é usado para referenciar sua versão de 16-bit. Esses quatro (EAX, EBX, ECX, EDX) podem ser referenciados como 8-bit, possuindo o AH (High) e AL (Low).

3.1.1 General Registers

Esses são usados para armazenar dados ou endereços de memória, porém também podem ser usados de forma intercalada para obter algum resultado. Aqui vale ressaltar que não é porque esses são chamados registradores de uso geral que eles são usados de qualquer maneira, isso varia por conta da convenção que cada compilador usa.

O importante aqui é que o **uso de registradores de forma consistente em todo o código compilado é conhecido como uma convenção. O conhecimento das convenções usadas pelos compiladores permite que um engenheiro reverso examine o código mais rapidamente, porque não se perde tempo tentando entender o contexto de como um registrador está sendo usado.** Por exemplo, normalmente o EAX geralmente contém o valor de retorno das chamadas de função. Portanto, se você vir o registrador EAX usado imediatamente após uma chamada de função (**call**), provavelmente verá o código manipulando o valor de retorno.

3.1.2 Segment Registers

Esse são usados para controlar a forma com que a memória é acessada e organizada pela CPU. Digamos que você está analisando um código malicioso, como dito antes, alguns desses possuem *tricks* para dificultar nossa análise. Um registrador comumente utilizado em sistemas Windows para validar se o programa está sendo executado sobre um depurador (*debugger*) é o **FS**. Pois quando um processo está em execução a PEB (Process Environment Block) pode ser referenciado usando o registrador FS. De forma resumida a PEB é uma estrutura de dados do Windows que contém informações sobre o Processo em execução, cada processo terá sua própria PEB.

```
mov eax, dword ptr fs:[30]
mov ebx, byte ptr [eax+2]
test ebx, edx
jz NoDebuggerDetected
```

Código-fonte 1 – Exemplo de validação do membro BeingDebugged
Fonte: Elaborada pelo autor (2023)

As instruções acima são um exemplo de uso do registrador **FS** para validar se o binário está sendo executado sobre um *debugger* olhando para o membro (BeingDebugged) no offset 2 da PEB. Quando você estiver utilizando o x64dbg ele irá ressaltar esses registradores para ajudá-lo em suas análises.

3.1.3 EFLAGS Registers

Esse são usados para operações aritméticas e tomada de decisões. Por exemplo, digamos que você esteja depurando um simples programa que pede o input de uma senha de no máximo 8 caracteres. Caso você coloque 5 caracteres, o programa irá validar se o tamanho que foi imputado satisfaz as necessidades do programa. Nesse caso, o programa irá mostrar uma mensagem de erro dizendo que a senha tem que ser no mínimo 8 caracteres. Porém é possível burlar essa validação, alterando o valor do registrador ZF (Zero Flag). Só para deixar claro, isso é só um exemplo, se eventualmente você se deparar com algo assim, isso indica um péssimo designe por parte do desenvolvedor.

3.1.4 EIP Register

O EIP (ou *Instruction Pointer*) é o registrador que aponta para a próxima instrução a ser executada pelo processador. Posteriormente veremos que quando o EIP é comprometido um atacante consegue manipulá-lo e apontar para um endereço de memória onde contém um *shellcode* malicioso.

3.2 Arithmetic Instructions

As instruções de aritméticas na linguagem assembly podem ser identificadas pelos mnemônicos listados na Tabela “Instruções de Adição, Subtração e incrementação”.

Instrução	Descrição
sub eax, 0x0A	Subtrai 10 do valor armazenado em EAX
add eax, ebx	Soma o valor de EBX com EAX e armazena o resultando em EAX
inc edx	Incrementa o valor de EDX por 1
dec ecx	Decrementa o valor de ECX por 1

Tabela 5 – Instruções de adição, subtração e incrementação
Fonte: Elaborada pelo autor (2023)

Vamos ver um exemplo de como isso é representando em Assembly, utilizando um simples código em C. **Por questões didáticas eu precisei desabilitar a opção de otimização do Visual Studio.** Porém no mundo real, você muito provável encontrar código muito mais enxutos por conta da otimização feita pelo compilador.

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 20;
    int result = 0;

    result = a + b;
    printf_s("A + B = %d\n", result);

    result = b - a;
    printf_s("B - A = %d\n", result);

    return 0;
}
```

Código-fonte 2 – Exemplo de cálculo em C
Fonte: Elaborada pelo autor (2023)

Esse simples código nos ajudará a entender o funcionamento e a interpretação de operações matemáticas básicas em Assembly.

```

mov     [ebp+var_A], 0Ah ; Inicializando a variável A com o valor 10.
mov     [ebp+var_B], 14h ; Inicializando a variável B com o valor 20.
mov     dword ptr [ebp+var_result], 0 ; Inicializando a variável result com o valor 0.
mov     eax, [ebp+var_A]
add     eax, [ebp+var_B]
mov     dword ptr [ebp+var_result], eax
mov     ecx, dword ptr [ebp+var_result]
push    ecx                ; char
push    offset Format      ; "A + B = %d"
call    printf_s

```

Figura 4 - Adição em Assembly
Fonte: Elaborada pelo autor (2023)

Então vamos com calma para que você não ficar confuso. Nas três primeiras instruções **MOV** estamos inicializando nossas três variáveis com os seus devidos valores. Logo em seguida vemos a sequência de instruções que realizam a operação de soma.

```

mov     eax, [ebp+var_A]
add     eax, [ebp+var_B]
mov     dword ptr [ebp+var_result], eax

```

Código-fonte 3 – Operação de soma em Assembly
Fonte: Elaborada pelo autor (2023)

Vamos entender o que está acontecendo. Primeiro o valor da *variável A* é movido para o registrador **EAX**. Em seguida o valor da *variável A* e *B* será somado utilizando a instrução **ADD** e o resultado dessa soma será armazenado no registrador **EAX**. E por último o valor de **EAX** é movido para a nossa *variável result*. Não foi tão difícil, foi? No final, o valor de *result* será passado como parâmetro para a função **printf_s** para mostrar o resultado da soma na tela.

```

mov     edx, [ebp+var_B]
sub     edx, [ebp+var_A]
mov     dword ptr [ebp+var_result], edx
mov     eax, dword ptr [ebp+var_result]
push    eax                ; char
push    offset aBAD        ; "B - A = %d\n"
call    printf_s

```

Figura 5 - Subtração em Assembly
Fonte: Elaborada pelo autor (2023)

Agora na figura “Subtração em Assembly”, vemos como é performedo a operação de subtração em Assembly. Não é muito diferente da vista anteriormente. A única coisa que muda é a instrução **ADD** para **SUB** e que estamos somando $B + A$

para não resultar em um valor negativo e o valor será armazenado no registrador EDX. Porém, a lógica é bem similar.

```
#include <stdio.h>

int main()
{
    int x = 10;

    __asm {
        mov eax, x
        inc eax
        mov x, eax
    };

    printf_s("Incremented value: %d\n", x);

    return 0;
}
```

Código-fonte 4 – Incrementando valor por 1
Fonte: Elaborada pelo autor (2023)

No código acima estamos utilizando um recurso do Visual Studio que nos permite adicionar instruções de Assembly no nosso código em C/C++. Isso é chamado de Inline Assembly. Quais as vantagens disso, muitas, pois dessa forma conseguimos otimizar ainda mais nosso código. Isso também é muito útil para performar syscalls. De forma resumida, uma syscall é uma chamada direta para o kernel do sistema, dessa forma em sistemas Windows não necessitando das Win32 APIs. Isso é muito utilizado por atores maliciosos para burlar antivírus e EDR. Mas vamos voltar ao assunto principal...

```
mov     dword ptr [ebp+var_X], 0Ah
mov     eax, dword ptr [ebp+var_X]
inc     eax
mov     dword ptr [ebp+var_X], eax
mov     eax, dword ptr [ebp+var_X]
push    eax                ; char
push    offset Format      ; "Incremented value: %d\n"
call    printf_s
```

Figura 6 - Incrementando valor por 1 em Assembly
Fonte: Elaborada pelo autor (2023)

Nesse nosso código estamos inicializando a *variável X* com o valor 10 (ou 0x0A em hexadecimal) em seguida movemos o valor para o registrador **EAX** e usamos a instrução **INC** para incrementar o valor por um. Para decrementar basta alterar a instrução **INC** para **DEC**.

```

mov     [ebp+var_4], eax
mov     [ebp+var_X], 0Ah
mov     eax, [ebp+var_X]
dec     eax
mov     [ebp+var_X], eax
mov     eax, [ebp+var_X]
push    eax
push    offset _Format ; "Decrementated value: %d\n"
call    _printf_s

```

Figura 7 - Decrementando valor por 1 em Assembly
 Fonte: Elaborada pelo autor (2023)

Ok, mas e a multiplicação e a divisão? Bom essas duas operações são performadas de forma diferente. Isso pois essas operações precisam de dois registradores em específico o **EAX** e o **EDX**.

As duas instruções para trabalhar com essas operações são:

Instrução	Descrição
mul 0x50	Multiplica EAX por 0x50 e armazena o resultado em EDX:EAX
div 0x75	Divide o valor de EDX:EAX por 0x75 e o resultado é armazenado em EAX e o restante em EDX

Tabela 6 – Instruções para multiplicação e divisão
 Fonte: Elaborada pelo autor (2023)

A instrução **mul valor** sempre irá multiplicar **EAX** pelo **valor**. Por conta disso o registrador **EAX** deve ser configurado apropriadamente antes que a multiplicação ocorra. O valor do resultado é armazenado como 64-bit, por conta disso precisamos de dois registradores para armazenar esse valor.

A instrução **div valor** fará a basicamente a mesma coisa só que na direção oposta. Nesse caso é dividido o valor de 64-bit entre o registrador **EDX** e **EAX** pelo **valor**. Vamos ver um exemplo para ficar mais claro.

```

#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = 2;
    int result = 0;

    result = a / b;
    // result = a * b;

    printf_s("A / B = %d\n", result);
    // printf_s("A * B = %d\n", result);

    return 0;
}

```

Código-fonte 5 – Exemplo de divisão e multiplicação em C
 Fonte: Elaborada pelo autor (2023)

```

mov     [ebp+var_A], 0Ah
mov     [ebp+var_B], 2
mov     dword ptr [ebp+var_result], 0
mov     eax, [ebp+var_A]
cdq
idiv     [ebp+var_B]
mov     dword ptr [ebp+var_result], eax
mov     eax, dword ptr [ebp+var_result]
push    eax                ; char
push    offset Format      ; "A / B = %d\n"
call    printf_s

```

Figura 8 - Divisão em Assembly
Fonte: Elaborada pelo autor (2023)

Novamente, vamos entender o que está acontecendo nesse bloco de instruções. Agora sem os comentários, nos três primeiros **MOV**s estamos inicializando nossas variáveis (*a*, *b* e *result*) com os seus devidos valores (10, 2 e 0). O valor da *variável A* é movido para **EAX**. A instrução **CDQ** (ou Convert Doubleword to Quadword) irá estender o bit de sinal de **EAX** para o registrador **EDX**.

Em seguida a instrução **IDIV** (aqui vale ressaltar que, a instrução **IDIV** e **IMUL** são as versões *signed* de **MUL** e **DIV**) irá realizar a divisão entre os dois registradores pelo **valor** armazenado na *variável B*. Por fim, salvando o resultando na *variável result* e mostrando o valor na tela.

Agora no caso da multiplicação vemos que o *variável A* é movida para **EAX** e é feito a multiplicação entre a *variável A* e *B*.

```

mov     [ebp+var_A], 0Ah
mov     [ebp+var_B], 2
mov     dword ptr [ebp+var_result], 0
mov     eax, [ebp+var_A]
imul    eax, [ebp+var_B]
mov     dword ptr [ebp+var_result], eax
mov     ecx, dword ptr [ebp+var_result]
push    ecx                ; char
push    offset Format      ; "A / B = %d\n"
call    printf_s

```

Figura 9 - Multiplicação em Assembly
Fonte: Elaborada pelo autor (2023)

Já as operações lógicas como (**AND**, **OR**, **XOR**) são realizadas de forma similar as instruções **ADD** e **SUB**. Performando a operação entre o operador de origem e destino, armazenando o resultando no operador de destino. Por exemplo, quando você encontrar uma sequência como **xor eax, eax**. Essa operação é uma forma rápida para limpar o registrador. Essa é uma forma otimizada pois essa instrução requer apenas 2 bytes, enquanto **mov eax, 0** necessita de 5 bytes.

Instruções como (**SHR** e **SHL**) são usadas para realizar um shift nos registradores. Sua sintaxe é **shr destino, contador**. **shl** segue a mesma sintaxe. O uso dos shifts é comumente usado como uma forma de otimizar a multiplicação, evitando a necessidade de mover dados para registradores. Já instruções como (**ROR** e **ROL**) são usados para rotacionar os valores, sua sintaxe é similar à do shift. Caso você encontre um bloco contendo muitas instruções de SHR, SHL, ROR, ROL, XOR, AND é muito provável que você estará lidando com alguma criptografia.

Abaixo podemos ver alguns exemplos dessas instruções.

Instruções	Descrição
xor eax, eax	Realiza a limpeza do registrador EAX.
or eax, 0x7575	Performa uma operação lógica em EAX com o valor 0x7575.
mov eax, 0xA shl eax, 2	Performa um shift de 2 bits a esquerda usando o registrador EAX. Resultando no valor 0x28, pois 1010 (0xA em binário) shifted por 2 bits a esquerda é 101000 (0x28).
mov bl, 0xA ror bl, 2	Rotaciona o registrador BL (de 8-bit) 2 bits a direita. O resultado final é BL = 10000010, pois 1010 rotacionado por 2 bits a direita é 10000010.

Tabela 7 – Exemplos comuns de instruções aritméticas lógicas e *Shifting*
Fonte: Elaborada pelo autor (2023)

3.3 Pilha (Stack)

Antes de entendermos sobre como é feito as chamadas de funções em assembly, precisamos entender, como a Stack (ou pilha) funciona.

A memória para funções, variáveis locais, e controle de fluxo são armazenadas na Stack, a pilha nada mais é que uma estrutura de dados que realizar PUSH (adiciona valores) e POP (retira valores). A stack opera como LIFO (*last in first out*).

Pense da seguinte forma, digamos que sua mãe peça para você guardas os pratos que foram lavados. Esses pratos estão organizados em uma pilha em cima da mesa. O primeiro prato que foi adicionado nessa pilha, será o último a ser guardado.



Figura 10 - Exemplo da *Stack*
Fonte: JAVAPOINT (s. d.)

A arquitetura x86 possui registradores que suportam operações com a *Stack* como o **ESP** e o **EBP**. O registrador **ESP** também conhecido como Stack Pointer (ou ponteiro da pilha) contém o endereço de memória que aponta para o topo da *Stack*. O valor desse registrador altera de forma com que valores são adicionados e removidos da *Stack*. Já o registrador **EBP** também conhecido como Base Pointer (ou ponteiro base) permanece consistente dentro de uma determinada função, para que o programa possa usá-lo como um espaço reservado para acompanhar a localização de variáveis e parâmetros locais.

Algumas das instruções relacionadas com a operação da *Stack* são:

Instruções	Descrição
push 0x0B	Adiciona o valor 11 (em decimal) na pilha.
pop 0x0B	Retira o valor 11 (em decimal) da pilha.
call 0x402921	Faz a chamada da função localizada no endereço 0x402921.
enter & leave	Essas instruções são usadas para criar e liberar automaticamente, e respectivamente o quadro da pilha para os procedimentos chamados.
ret	Retorno de função

Tabela 8 – Instruções da *Stack*
Fonte: Elaborada pelo autor (2023)

3.4 Function Calls

As funções são pequenos blocos de códigos que performam uma determinada ação como por exemplo a soma de valores. Agora vamos dar entrada um pouco mais a fundo nos detalhes com base no código abaixo:

```
#include <stdio.h>

int check(int a, int b)
{
    if (a > b)
    {
        printf_s("A is bigger than B\n");
        return a;
    }
    else if (a < b)
    {
        printf_s("B is bigger than A\n");
        return b;
    }
    else
    {
        printf_s("The values are equal\n");
        return 0;
    }
}

int main(void)
{
    int A = 0;
    int B = 0;
    int result = 0;

    printf_s("Input the first value: ");
    scanf_s("%d", &A);
    printf_s("Input the second: ");
    scanf_s("%d", &B);

    result = check(A, B);

    if (result != 0)
    {
        printf_s("The biggest value is: %d\n", result);
    }

    return 0;
}
```

Código-fonte 6 – Exemplo de função
Fonte: Elaborada pelo autor (2023)

Bom, vamos entender o que esse código está fazendo e ao mesmo tempo vamos reconhecer suas construções em na linguagem assembly.

Se você não está acostumado ou nunca programou na linguagem C, não tem problema, irei dar uma breve pincelada em certos pontos. Acho que você já reparou que todos programas que utilizamos começa com uma função **main()**. Essa é a primeira função a ser executada nos programas. Na verdade, eu estou mentindo. A primeira função a ser executada é o Entry Point do programa. Falaremos

posteriormente sobre isso. Porém, a primeira função do nosso programa que contém o nosso código é a `main`. Vamos quebrá-la em parte:

1. **int** – Esse é o valor de esperado do retorno da função.
2. **main** – É o nome da função.
3. **(void)** – Indica que essa função não recebe nem um argumento. Na maioria das vezes você irá encontrar **`main(int argc, char* argv[])`**.
4. Tudo entre `{ ... }` são as operação realizada pela função.

Ok, agora que entendemos isso podemos começar a falar sobre o *prologue* (ou prólogo) e o *epilogue* (ou epílogo). O prólogo são as primeiras linhas de instruções que são usadas para inicializar a função.

```
push    ebp
mov     ebp, esp
sub     esp, 10h
```

Figura 11 - Prólogo da função `main`
Fonte: Elaborada pelo autor (2023)

Essas três instruções estão inicializando a função **`main()`** do nosso programa. Basicamente o prólogo irá preparar a Stack e os registradores que serão utilizados pela função.

```
mov     esp, ebp
pop     ebp
retn
```

Figura 12 – Epílogo da função `main`
Fonte: Elaborada pelo autor (2023)

Já o epílogo representa o final da função. Ele é responsável por restaurar a Stack e os registradores para o seu estado antes da função ser chamada.

```
push    ebp
mov     ebp, esp
sub     esp, 10h
mov     eax, __security_cookie
xor     eax, ebp
mov     [ebp+var_4], eax
mov     dword ptr [ebp+var_A], 0
mov     dword ptr [ebp+var_B], 0
mov     dword ptr [ebp+var_result], 0
```

Figura 13 – Inicializando as variáveis (A, B e *result*)
Fonte: Elaborada pelo autor (2023)

Na figura “Inicializando as variáveis (A, B e *result*)” vemos o prólogo da função `main` sendo inicializado seguido pelo `__security_cookie`, você não precisa se

preocupar com isso agora. Em seguida nossas 3 variáveis são inicializadas com o valor zero. Já vimos isso antes né?

```

push    offset aInputTheFirstV ; "Input the first value: "
call    _printf_s
add     esp, 4
lea     eax, [ebp+var_A]
push    eax                    ; char
push    offset aD              ; "%d"
call    _scanf_s
add     esp, 8
push    offset aInputTheSecond ; "Input the second: "
call    _printf_s
add     esp, 4
lea     ecx, [ebp+var_B]
push    ecx                    ; char
push    offset aD_0            ; "%d"
call    _scanf_s

```

Figura 14 – Instruções e chamadas de função para receber o *input* do usuário
Fonte: Elaborada pelo autor (2023)

Após a inicialização das variáveis vemos a sequência de chamadas para as funções **printf_s()** e **scanf_s()** que será responsável por exibir uma mensagem na tela e receber o input do usuário.

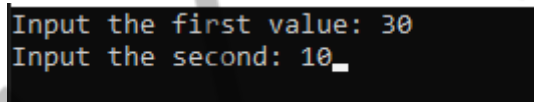


Figura 15 – Exemplo de *input* do usuário no cmd
Fonte: Elaborada pelo autor (2023)

Na figura “Exemplo de *input* do usuário no cmd” vemos como a mensagem é mostrada na tela do usuário e o input dos valores pelo usuário.

```

mov     edx, dword ptr [ebp+var_B]
push    edx
mov     eax, dword ptr [ebp+var_A]
push    eax
call    check

```

Figura 16 – Parâmetros sendo jogados na *Stack* e chamada para nossa função **check()**
Fonte: Elaborada pelo autor (2023)

Na figura “Parâmetros sendo jogados na *Stack* e chamada para nossa função **check()**” podemos ver que os valores que foram passados pelo usuário serão jogadas na *Stack* para que possam ser tratado pela função **check()** que será posteriormente executada. Essa nossa função irá validar qual dos valores é o maior.

Vamos fazer um simples overview um pouco detalhado sobre o que acontece nas chamadas de funções.

1. Primeiro os argumentos necessários da função são Pushed (jogados) na Stack.
2. A função então é chamada da seguinte forma **call <endereço>** (no nosso caso eu dei um nome para esse endereço). Isso faz com que o endereço da instrução atual (ou seja, o conteúdo do registrador **EIP**) seja jogado na pilha. Esse endereço será utilizado para o retorno ao código da função **main()** quando a função for finalizada.
3. Através do uso de um prólogo de função, o espaço é alocado na pilha para variáveis locais e o **EBP** (o ponteiro base) é colocado na pilha. Isso é feito para economizar **EBP** para a função de chamada.
4. A função faz o que tem que fazer.
5. Através do uso do epílogo da função, a Stack é recuperada. O **ESP** é ajustado para liberar as variáveis locais e o **EBP** é restaurado para que as funções de chamada possam endereçar suas variáveis adequadamente. A instrução **LEAVE** pode ser usada como um epílogo porque define ESP para igualar EBP e poped (retira) EBP da pilha.
6. A função retorna chamando a instrução **RET**. Isso remove o endereço de retorno da pilha e o coloca no **EIP**, de modo que o programa continuará executando de onde a chamada original foi feita.
7. Por fim, a Stack é ajustada para remover os argumentos que foram jogados, a menos que sejam usados novamente posteriormente.

Ok, agora que entendemos como uma chamada de função funciona vamos entender sobre as condições aplicadas nesse nosso simples código.

3.5 Conditionals & Branching

Todas as linguagens de programação possuem a capacidade de realizar comparações e tomadas de decisão, isso é claro com base na lógica desenvolvida pelo programador. As condições são instruções que performam algum tipo de comparação. As instruções mais comuns relacionadas a essa operação são: **TEST** e **CMP**.

Já uma Branch é uma sequência de instruções que é executada dependendo da forma como o programa flui. As Branchs ocorrem com uma instrução **JMP** (salto). Sua sintaxe é bem simples como se pode ver, **jmp <local>**.

Vamos agora entender nossa função **check()**.

```
.text:00401100      push     ebp                ; char
.text:00401101      mov      ebp, esp
.text:00401103      mov      eax, [ebp+arg_a]
.text:00401106      cmp      eax, [ebp+arg_b]
.text:00401109      jle      short loc_40111F
.text:0040110B      push     offset Format      ; "A is bigger than B\n"
.text:00401110      call     printf_s
.text:00401115      add      esp, 4
.text:00401118      mov      eax, [ebp+arg_a]
.text:0040111B      jmp      short loc_40114A
.text:0040111D      ; -----
.text:0040111D      jmp      short loc_40114A
.text:0040111F      ; -----
.text:0040111F      loc_40111F:                ; CODE XREF: check+9↑j
.text:0040111F      mov      ecx, [ebp+arg_a]
.text:00401122      cmp      ecx, [ebp+arg_b]
.text:00401125      jge      short loc_40113B
.text:00401127      push     offset aBIsBiggerThanA ; "B is bigger than A\n"
.text:0040112C      call     printf_s
.text:00401131      add      esp, 4
.text:00401134      mov      eax, [ebp+arg_b]
.text:00401137      jmp      short loc_40114A
.text:00401139      ; -----
.text:00401139      jmp      short loc_40114A
.text:0040113B      ; -----
.text:0040113B      loc_40113B:                ; CODE XREF: check+25↑j
.text:0040113B      push     offset aTheValuesAreEq ; "The values are equal\n"
.text:00401140      call     printf_s
.text:00401145      add      esp, 4
.text:00401148      xor      eax, eax
.text:0040114A      loc_40114A:                ; CODE XREF: check+1B↑j
.text:0040114A      ; check+1D↑j ...
.text:0040114A      pop      ebp
.text:0040114B      retn
.text:0040114B      check
.text:0040114B      endp
```

Figura 17 – Função check()
Fonte: Elaborada pelo autor (2023)

A branch mais popular que você encontrará será a **JMP**. Essa instrução simplesmente irá pular para o endereço para o qual aponta. Vamos entender detalhadamente.

```
push     ebp                ; char
mov      ebp, esp
mov      eax, [ebp+arg_a]
cmp      eax, [ebp+arg_b]
jle      short loc_40111F
push     offset Format      ; "A is bigger than B\n"
call     printf_s
add      esp, 4
mov      eax, [ebp+arg_a]
jmp      short loc_40114A
```

Figura 18 – Primeiro bloco
Fonte: Elaborada pelo autor (2023)

Nas duas primeira linhas temos o prólogo da função **check()**. Logo em seguida nosso *argumento A* é movido para **EAX** e **EAX** é comparado com o *argumento B*. **JLE** (*Jump if Less or Equal*) é uma variação da instrução **JMP**. Nesse caso estamos validando se o valor de **A** é menor que o valor de **B** caso seja ele irá pular para nossa segunda condição (*loc_40111F*). Caso **A** seja maior que **B** então o programa irá imprimir na tela a mensagem **A is bigger than B** e o programa irá terminar sua execução.

```
mov     ecx, [ebp+arg_a]
cmp     ecx, [ebp+arg_b]
jge     short loc_40113B
push    offset aBIsBiggerThanA ; "B is bigger than A\n"
call    printf_s
add     esp, 4
mov     eax, [ebp+arg_b]
jmp     short loc_40114A
```

Figura 19 – Segundo bloco
Fonte: Elaborada pelo autor (2023)

Já o segundo bloco, só será executado se a primeira validação falhar. Ou seja, se **B** for maior que **A**. Nesse caso o programa irá imprimir a mensagem **B is bigger than A**.

Tanto na figura “Primeiro bloco” como na figura “Segundo bloco”, após a chamada da função **printf_s()**, o valor o maior valor é movido para **EAX** como o valor de retorno.

```
push    offset aTheValuesAreEq ; "The values are equal\n"
call    printf_s
add     esp, 4
xor     eax, eax
```

Figura 20 – Terceiro bloco
Fonte: Elaborada pelo autor (2023)

Se caso nem uma das condições anteriores for verdade então o programa irá assumir que os dois valores são iguais e exibir a mensagem **The values are equal**. Retornando zero como valor.

Uma dica muito importante para se interpretar bem o assembly é sempre ler as instruções por blocos. Mas como assim? Você precisa interpretar as instruções em um contexto, e não perder tempo tentando entender o que cada instrução está fazendo. Um exemplo claro é a instrução **ADD ESP, 4**. Eu não cheguei a comentar dela em nem um dos blocos anteriores. Não porque ela não tem um

significado para a operação do nosso programa, mas simplesmente pois não precisamos necessariamente entender o que cada instrução está fazendo para compreender a lógica do programa.

Mas só para matar a curiosidade e mostrar que essa instrução possui um significado, ela simplesmente está limpando os argumentos da função **printf_s()**. Isso é por conta da convenção de chamada da função. Veremos sobre isso a seguir.

3.6 Call Conventions

Convenções de chamadas (ou *Call Conventions*) são usadas por funções nos programas e dependendo da convenção usada isso pode interferir levemente em como as instruções são representadas no *dissassembler*. Cada convenção possui sua forma de passar os argumentos para *stack* ou os registradores usados e se vai ser o caller ou a função que for chamada (*callee*) que irá limpar a Stack a pós o uso da função.

Existem outras convenções como **__thiscall**, **__vectorcall** e **__clrcall**. Porém só abordaremos as mais comuns utilizadas.

3.6.1 Cdecl

Essa é a convenção mais popular (como vimos acima), pois ela é a convenção padrão das linguagens C e C++. Nessa convenção de chamada, os parâmetros são jogados na Stack da direita para esquerda, o caller (ou seja, quem chama a função) é responsável por limpar a Stack após a finalização da função e o valor de retorno é armazenada no registrador **EAX**. Exatamente como vimos na figura “Função check()”.

Função main		Função add	
push	ebp	push	ebp
mov	ebp, esp	mov	ebp, esp
sub	esp, 0Ch	mov	eax, [ebp+arg_0]
mov	[ebp+var_A], 0Ah	add	eax, [ebp+arg_4]
mov	[ebp+var_B], 14h	pop	ebp
mov	dword ptr [ebp+var_result], 0	retn	
mov	eax, [ebp+var_B]		
push	ecx, [ebp+var_A]		
push	ecx		
call	add		
add	esp, 8		
mov	dword ptr [ebp+var_result], eax		
mov	edx, dword ptr [ebp+var_result]		
push	edx ; char		
push	offset Format ; "Value: %d\n"		
call	printf_s		
add	esp, 8		
xor	eax, eax		
mov	esp, ebp		
pop	ebp		
retn			

Figura 21 – Exemplo de convecção de chamada `__cdecl`
 Fonte: Elaborada pelo autor (2023)

Como podemos ver na figura “Exemplo de convecção de chamada `__cdecl`”, nossa função **main()** (atuando como o caller, realiza a chamada para a função **add()**, passando como argumento as variáveis *A* e *B*. Após a função **add()** realizar sua operação vemos que a instrução **add esp, 8** faz a limpeza da Stack. E por fim o valor do registrador EAX é salve na variável *result*.

3.6.2 Stdcall

A convenção de chamada `stdcall` é muito comum no Windows. Ele é usado por quase todas as APIs do Windows e funções do sistema. A diferença entre `stdcall` e `cdecl` é que as funções `stdcall` são responsáveis por limpar sua própria pilha, enquanto em `cdecl` isso é responsabilidade do chamador.

Função main		Função add	
push	ebp	push	ebp
mov	ebp, esp	mov	ebp, esp
sub	esp, 0Ch	mov	eax, [ebp+arg_0]
mov	[ebp+var_A], 0Ah	add	eax, [ebp+arg_4]
mov	[ebp+var_B], 14h	pop	ebp
mov	dword ptr [ebp+var_result], 0	retn	8
mov	eax, [ebp+var_B]		
push	ecx, [ebp+var_A]		
push	ecx		
call	add		
mov	dword ptr [ebp+var_result], eax		
mov	edx, dword ptr [ebp+var_result]		
push	edx ; char		
push	offset Format ; "Value: %d\n"		
call	printf_s		
add	esp, 8		
xor	eax, eax		
mov	esp, ebp		
pop	ebp		
retn			

Figura 22 – Exemplo de convecção de chamada `stdcall`
 Fonte: Elaborada pelo autor (2023)

Como podemos ver na função `add` (que está atuando como o `callee`) faz o uso da instrução `RETN` que recebe a quantidade de bytes a serem limpos da Stack. Isso significa que o valor passado para a instrução **RETN** indica a quantidade de bytes passados como parâmetros, sendo que um valor inteiro tem um tamanho de 4 bytes, como passamos 2 parâmetros do tipo `int` nossa função faz o uso de 8 bytes.

3.6.3 Fastcall

Como o próprio nome indicar essa é uma convenção que tem como objetivo ter uma alta performance. Nessa convenção de chamada os dois primeiros parâmetros são passados usando os registradores **ECX** e **EDX**, caso haja outros parâmetros esses são jogados na Stack.

O diagrama ilustra a convenção de chamada `fastcall` entre duas funções. À esquerda, o código da **Função main** prepara os parâmetros: empilha o endereço da função `add` em `ebp`, subtrai `0Ch` de `esp`, movimenta `0Ah` para `[ebp+var_A]`, `14h` para `[ebp+var_B]`, define um ponteiro para o resultado em `[ebp+var_result]`, movimenta `[ebp+var_B]` para `edx` e `[ebp+var_A]` para `ecx`, e então chama a função `add`. À direita, o código da **Função add** recupera o endereço de retorno de `ebp`, movimenta `esp` para `ebp`, subtrai 8 bytes de `esp`, movimenta `edx` para `[ebp+var_8]`, `ecx` para `[ebp+var_4]`, soma `[ebp+var_4]` a `eax`, soma `[ebp+var_8]` a `eax`, movimenta `ebp` para `esp` e retorna com `retn`. Uma seta indica o retorno da função `add` para a função `main`.

Figura 23 – Exemplo de convenção de chamada `fastcall`
Fonte: Elaborada pelo autor (2023)

Como podemos ver na função `main()`, os dois únicos parâmetros são passados usando os registradores **ECX** e **EDX**.

3.7 Loops

Laços de repetição (ou *loops*) também são tipos de condições só que são executados até uma certa condição deixar de ser verdade. Utilizamos os loops na programação para não ter que refazer novamente a mesma coisa. Pensa da seguinte forma, digamos que você queira imprimir na tela 10 números, de 0 a 9. Ao invés de usar 10 vezes a função `printf_s()` você pode usar um laço de repetição para interagir quantas vezes forem necessários com aquele bloco de código.

Vamos ver alguns exemplos de loops na linguagem C e como identificá-los em assembly.

3.7.1 FOR loops

Começando com o FOR loop, sua sintaxe é: *valor de início, condição, contador*. Abaixo podemos ver um simples código em C que faz o uso desse laço de repetição.

```
#include <stdio.h>

void main(void)
{
    int i;

    for (i = 0; i < 10; i++)
    {
        printf_s("%d\n", i);
    }
}
```

Código-fonte 7 – Exemplo de uso do laço de repetição FOR
Fonte: Elaborada pelo autor (2023)

Nesse simples código temos a seguinte lógica, iniciamos a variável *i* com o valor 0. Em seguida criamos uma condição que irá ditar quantas vezes esse laço de repetição irá interagir com o bloco de código entre { ... }. Por fim nosso contador que também é a variável *i* irá se autoincrementar a cada volta que o loop der.

Vamos ver como isso fica após ser compilado.

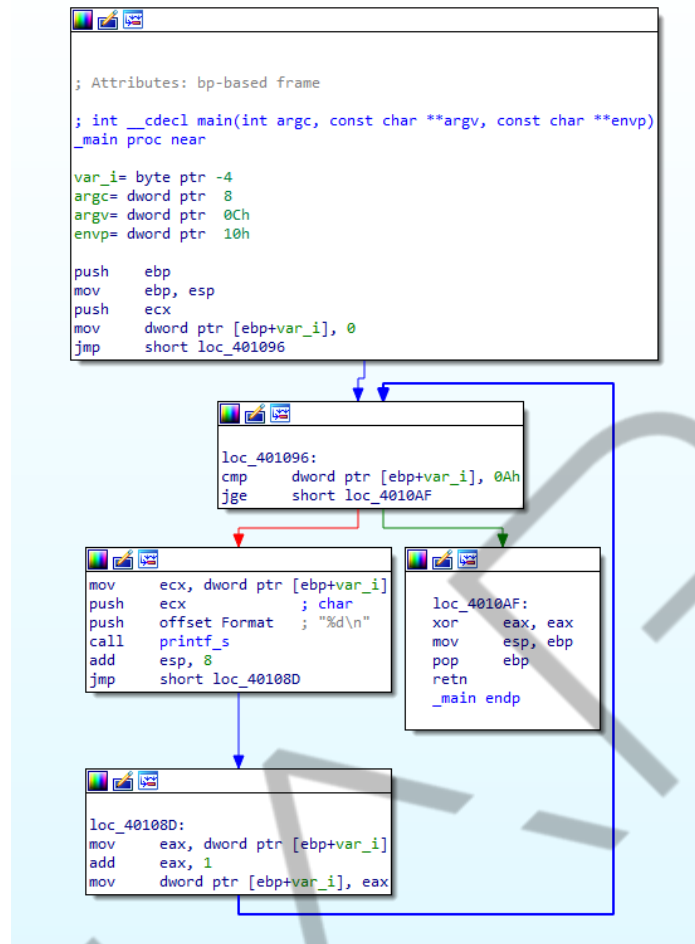


Figura 24 – Laço de repetição FOR na linguagem assembly
 Fonte: Elaborada pelo autor (2023)

Na figura “Laço de repetição FOR na linguagem assembly” estamos usando o modo gráfico do IDA, para conseguir ver nesse modo basta aperta a teclas de espaço do seu teclado.

No primeiro bloco nossa variável *i* é inicializada com o valor 0. Em seguida é feito uma comparação com o valor 10 (0x0A em hexadecimal) e entramos em uma condição que só será verdade se o valor de *i* for maior ou igual a 10 **JGE** (Jump if Greater or Equal). Caso contrário os números continuaram a ser exibidos na tela.

Podemos interpretar cada bloco da seguinte forma:

- O primeiro bloco contém nosso inicializador.
- O bloco localizado em loc_401096 é a nossa condição.
- O bloco com a seta vermelha é o bloco de código que será executado até que a condição se satisfaça.
- O bloco localizado em loc_40108D é o nosso contador.

- E o bloco localizado em `loc_4010AF` é a saída do programa.

3.7.2 While & Do While

Os laços de repetição FOR e WHILE são o que chamamos de Pretested loops (ou pré--testado) eles validam a condição antes de executar o bloco de código. Já o DO WHILE é um laço de repetição do tipo Posttested loops (ou pós-testado). Vamos ver alguns exemplos, e veremos que a diferença deles na linguagem assemble é relativamente pequena.

3.7.2.1 While

Abaixo podemos ver um simples código utilizando o laço de repetição while.

```
#include <stdio.h>

void main(void)
{
    int i = 0;

    while (i < 10)
    {
        printf_s("%d\n", i);
        i++;
    }
}
```

Código-fonte 8 – Exemplo de uso do laço de repetição FOR
Fonte: Elaborada pelo autor (2023)

No código “Exemplo de uso do laço de repetição FOR”, novamente inicializamos nossa variável `i` como nosso contador. E passamos a condição no começo do loop, enquanto `i` for menor que **10**, nosso bloco será executado exibindo os números na tela.

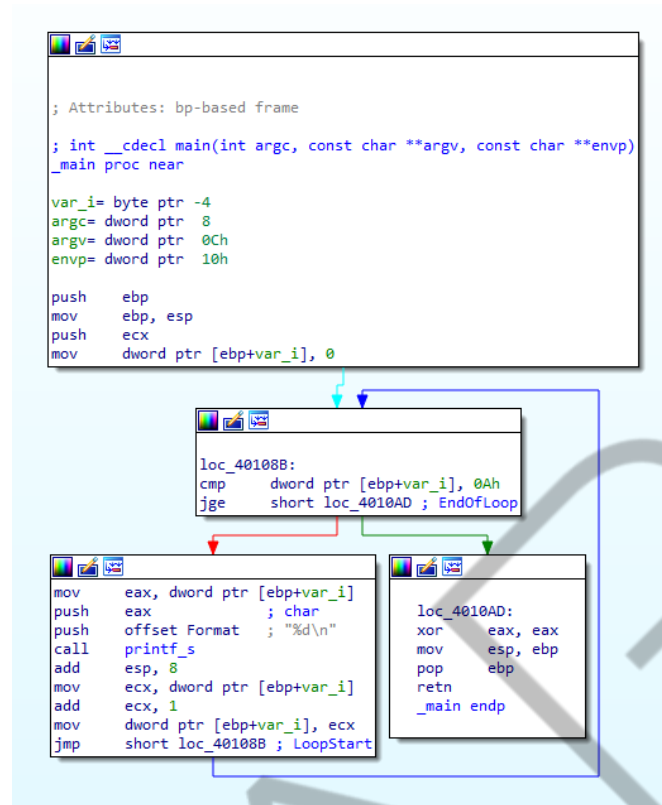


Figura 25 – Laço de repetição WHILE na linguagem assembly
 Fonte: Elaborada pelo autor (2023)

A diferença é quase que pequena comparado ao laço de repetição FOR. Primeiro nossa variável *i* é comparada como o valor 10, e só em seguida é inicializado nosso loop, usando o registrador **ECX** como o contador.

3.7.2.2 Do While

Como dito antes o DO WHILE é um laço de repetição do tipo pós-testel, o que significa que primeiro o nosso bloco de código é executado e depois é feito a validação. Abaixo podemos ver um simples exemplo.

```

#include <stdio.h>

int main()
{
    int i = 0;
    do {
        printf_s("%d\n", i);
        i++;
    } while (i < 10);

    return 0;
}

```

Código-fonte 9 – Exemplo de uso do laço de repetição DO WHILE
 Fonte: Elaborada pelo autor (2023)

A única diferença do código anterior é o laço de repetição em uso.

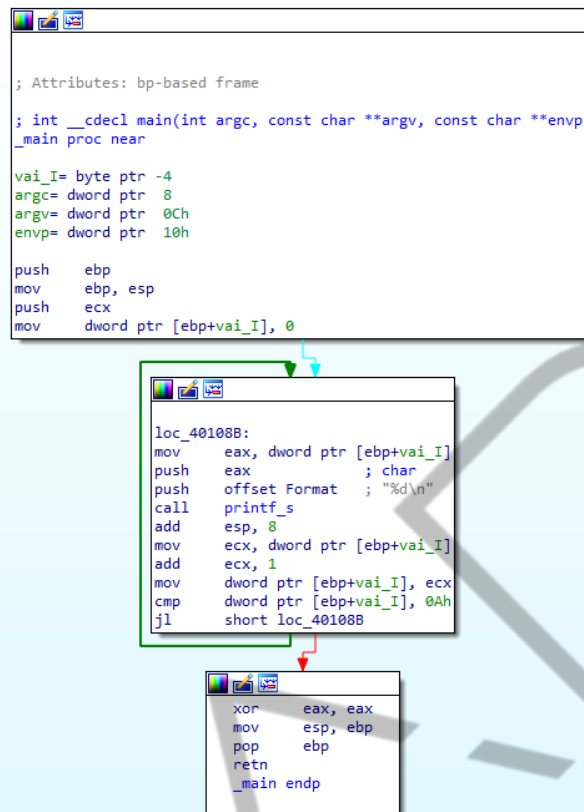


Figura 26 – Laço de repetição DO WHILE na linguagem assembly
Fonte: Elaborada pelo autor (2023)

Na figura “Laço de repetição DO WHILE na linguagem assembly” vemos um exemplo de como o laço de repetição DO WHILE é representado na linguagem assembly.

4 FERRAMENTAS

As ferramentas de engenharia reversa são de extrema importância, pois sem elas ficaria ainda mais difícil depurar, e interpretar o que o código de máquina está nos dizendo. Para isso ferramentas como *disassembler* e *debuggers* são muito úteis para nós. Aqui irei mencionar algumas das tools mais populares que podemos utilizar durante nossas análises.

4.1 IDA Pro

IDA (ou *Interactive Disassembler*) é um dos *disassemblers* mais poderosos atualmente desenvolvido pela Hex-rays. Essa ferramenta nos ajuda a entender

estaticamente a lógica de um binário. Como podemos ver na Figura “IDA Free disassembler”, vamos fazer um overview para nos familiarizar com a ferramenta.

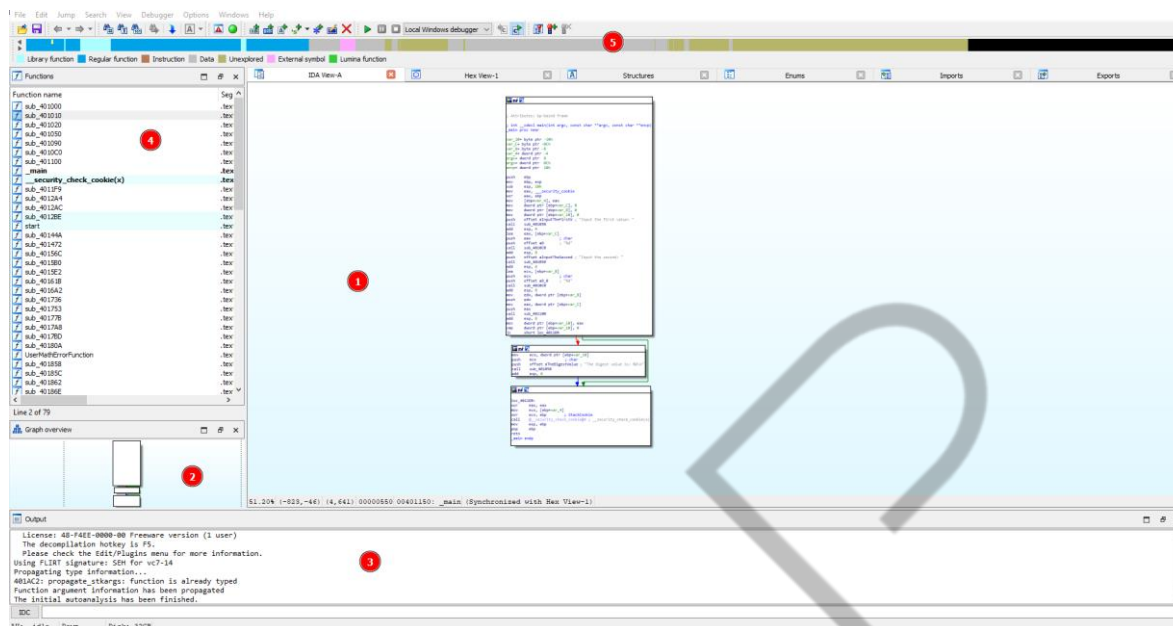
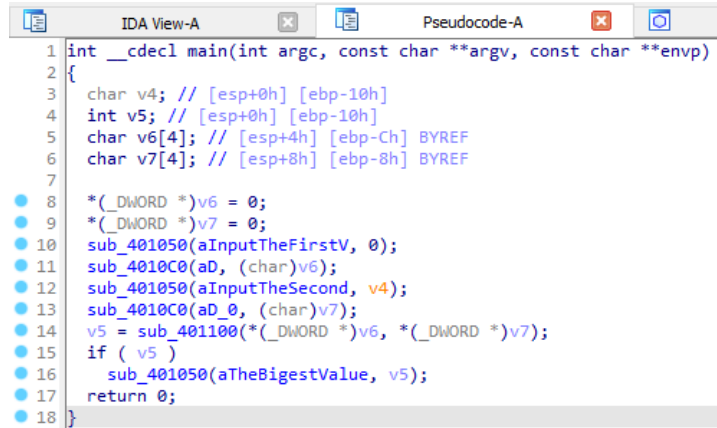


Figura 27 – IDA Free disassembler
Fonte: Elaborada pelo autor (2023)

Na posição 1 vemos a versão gráfica do disassembler que usamos bastando durante esse capítulo. Na posição 2 vemos um *overview* do gráfico que é mostrado no *disassmbler*, o que nos ajuda quando estamos lidando com um binário muito grande. Na posição 3 temos a console que nos permite executar alguns comandos e script para automatizar certas tarefas. Na posição 4 temos a funções que foram identificadas pelo Ida. Já na posição 5 temos uma barra que nos ajuda a visualizar os blocos onde contêm código, dados e o que o IDA não conseguiu identificar.

Existem diferentes versões do IDA, pois essa é uma ferramenta comercial e nada barata no ponto de vista do brasileiro. A versão que utilizamos foi a IDA Free, porém existem o IDA Pro, IDA Home, IDA Teams.

Essa ferramenta na versão gratuita possui um decompilador cloud-based. Um decompilador irá fazer o oposto de um compilador. Possibilitando visualizar uma representação em pseudocódigo.



```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     char v4; // [esp+0h] [ebp-10h]
4     int v5; // [esp+0h] [ebp-10h]
5     char v6[4]; // [esp+4h] [ebp-Ch] BYREF
6     char v7[4]; // [esp+8h] [ebp-8h] BYREF
7
8     *(_DWORD *)v6 = 0;
9     *(_DWORD *)v7 = 0;
10    sub_401050(aInputTheFirstV, 0);
11    sub_4010C0(aD, (char)v6);
12    sub_401050(aInputTheSecond, v4);
13    sub_4010C0(aD_0, (char)v7);
14    v5 = sub_401100(*(_DWORD *)v6, *(_DWORD *)v7);
15    if ( v5 )
16        sub_401050(aTheBigestValue, v5);
17    return 0;
18 }
```

Figura 28 – Exemplo de decompilação utilizando o IDAFree
Fonte: Elaborada pelo autor (2023)

Como podemos ver o decompiler irá tentar chegar ao mais próximo do código uma vez original.

4.2 Ghidra

Outra ferramenta muito poderosa e *open source* é o Ghidra. O Ghidra também é um *dissassembler* e decompiler desenvolvido pela NSA (National Security Agency). Essa é uma ferramenta que eu particularmente uso no dia a dia, porém eu decidi optar por usar o IDA nesse capítulo por considerar que ele é um pouco mais user-friendly de se usar.

O Ghidra é um pouco mais lento do que o IDA, pois ele foi desenvolvido em Java. Muitas pessoas acham o Ghidra difícil de usar, pois muitas delas não sabem instalá-lo corretamente. Vamos ver um simples tutorial de como se instalar corretamente o Ghidra no sistema Windows 10.

Primeiro precisamos fazer o download da última versão do Ghidra em seu próprio github.

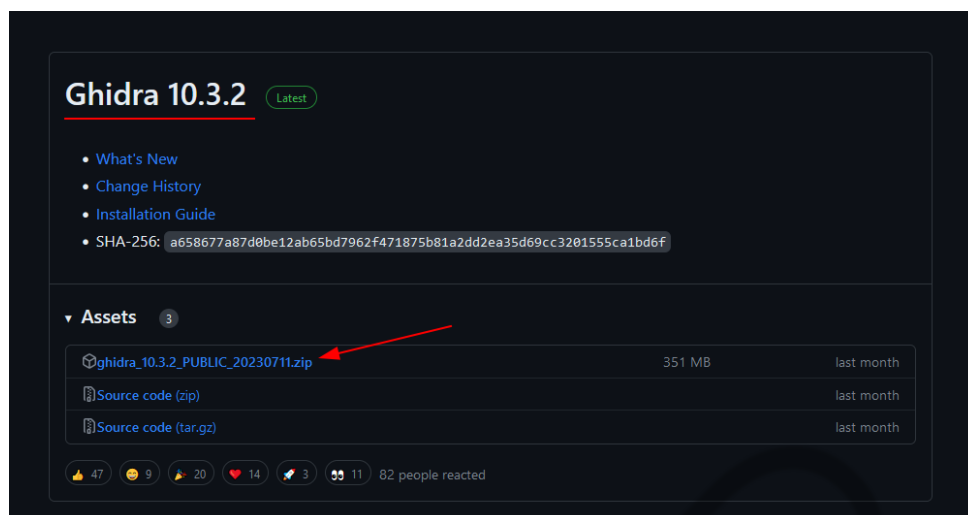


Figura 29 – Instalando o Ghidra
Fonte: Elaborada pelo autor (2023)

Faça o download do arquivo **ghidra_<versão>_PUBLIC_<data>.zip**. Após realizar o download do arquivo zip, precisamos fazer o download do JDK (Java Runtime and Development Kit) para conseguirmos usar corretamente o Ghidra. Nesse tutorial eu optei por utilizar a versão 17 do JDK.

Java 20 and Java 17 available now

JDK 20 is the latest release of Java SE Platform and JDK 17 LTS is the latest long-term support release for the Java SE platform.

[Learn about Java SE Subscription](#)

JDK 20 JDK 17 GraalVM for JDK 20 GraalVM for JDK 17

JDK Development Kit 17.0.8 downloads

JDK 17 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions](#).

JDK 17 will receive updates under these terms, until September 2024, a year after the release of the next LTS.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	172.38 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.zip (sha256)
x64 Installer	153.48 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.exe (sha256)
x64 MSI Installer	152.27 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi (sha256)

Figura 30 – Instalando o JDK
Fonte: Elaborada pelo autor (2023)

Após finalizar o download basta seguir como qualquer instalação do Windows (next, next, finished). Nosso próximo passo é adicionar os binários do Java em nossa variável de ambiente PATH. Para isso basta seguir as instruções abaixo:

1. Com o botão direito do mouse clique em *iniciar* e em seguida em *Sistema*.

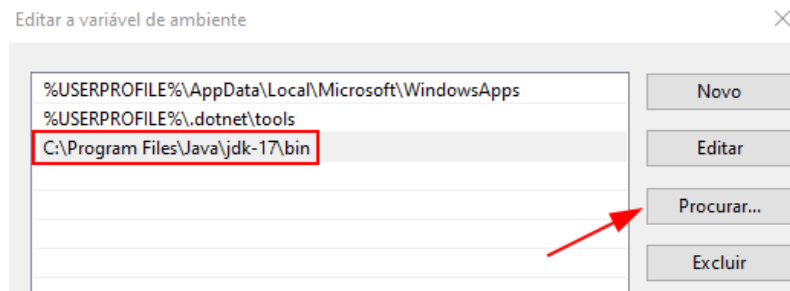


Figura 33 – Adicionando o path para os binários do JDK
Fonte: Elaborada pelo autor (2023)

5. Para finalizar basta dar **ok** três para salvar.

Agora com os binários do Java configurado corretamente podemos executar o arquivo .bat para inicializar o Ghidra.

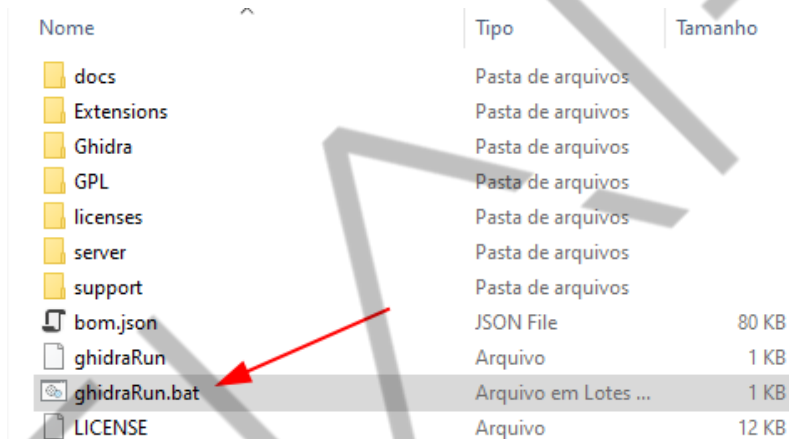


Figura 34 – Executando o Ghidra
Fonte: Elaborada pelo autor (2023)

Se você chegou até aqui, você fez tudo certo, e o Ghidra já pode ser usado para suas análises.

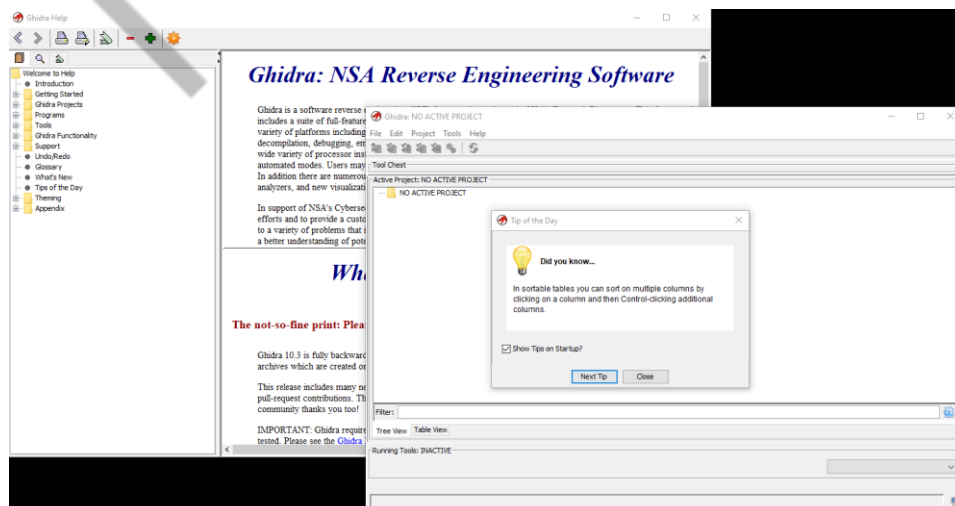


Figura 35 – Ghidra Instalado
Fonte: Elaborada pelo autor (2023)

Agora vamos criar um simples projeto para que possamos analisar um binário. Para isso vá em *File > Non-shared Project > [nome do projeto] > finished*. Em seguida importe o arquivo que você deseja analisar, para isso vá em *File > Import File >* e selecione o arquivo.

Após finalizar a importação do arquivo o Ghidra irá nos retornar um breve overview com algumas informações do binário.

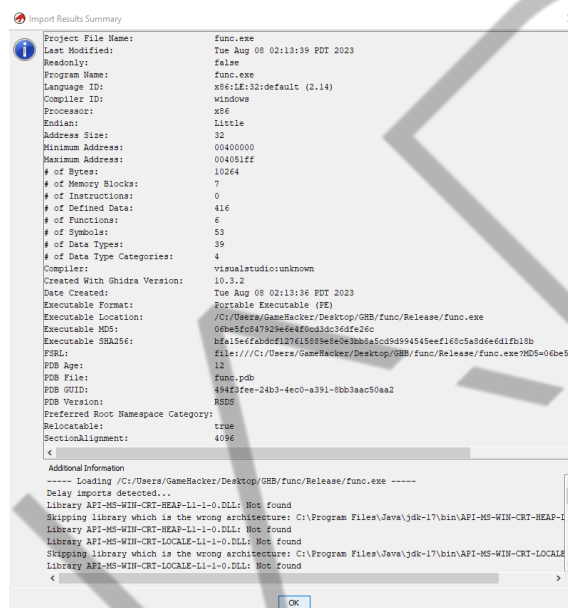


Figura 36 – Overview do binário a ser analisado usando o Ghidra
Fonte: Elaborada pelo autor (2023)

Para usar o disassembler clique no CodeBrowser.

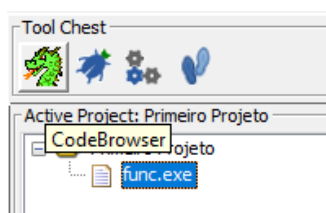


Figura 37 – CodeBrowser
Fonte: Elaborada pelo autor (2023)

Assim que o CodeBrowser abrir basta você abrir o arquivo importado anteriormente e permitir que o CodeBrowser o analise.

A primeira vista o Ghidra não é tão intuitivo quanto o IDA que já nos leva diretamente para a função **main()** do programa. As vezes o Ghidra requer um pouco de esforço por parte do analista.

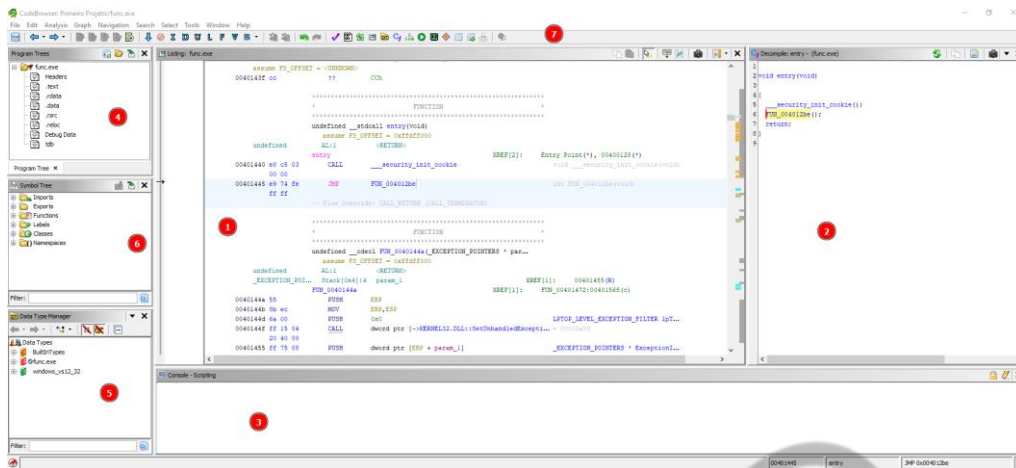


Figura 38 – Ghidra
Fonte: Elaborada pelo autor (2023)

Porém ele é bem similar ao IDA, na posição 1 temos o Listing que irá mostrar o disassembler do programa. Na posição 2 temos o decompiler do programa. Na posição 3 temos a console que nos permite executar alguns comandos e rodar alguns scripts em Python ou Java. Na posição 4, 5, 6 temos algumas informações sobre as seções do arquivo, import e exports do programa e o data type manager que armazena diferentes símbolos para ajudar na nomenclatura de certos parâmetros de função e até criação de novos tipos de dados.

Usar o Ghidra requer um pouco de esforço por parte do analista, porém é a opção mais barata por ser open-source, sua documentação é muito bem detalhada.

4.3 X64dbg

O x64dbg é um dos *debugger* de user-mode mais populares atualmente. Permitindo a criação de scripts/plugins que ajuda em diferentes áreas como exploração de binário e análise de malware.

A instalação desse *debugger* é bastante simples basta fazer o download da última versão disponível no github e extrair os arquivos para o local desejado.

Esse *debugger* vem com duas versões uma para 32-bit e outra para 64-bit. Abaixo usando o x32dbg podemos ver cada campo do nosso *debugger*.

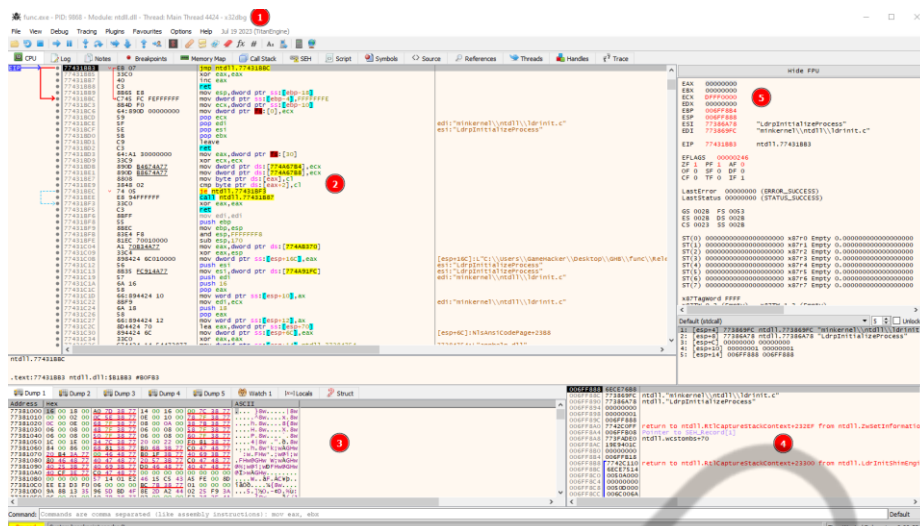


Figura 39 – x64dbg em sua versão 32-bit

Fonte: Elaborada pelo autor (2023)

Essa vai ser uma dica muito importante, pois assim que você inicia o *debugger* ele irá fazer uma pausa no modulo *ntdll.dll*. Ou seja, no *loader* do programa. Como podemos ver abaixo:



Figura 40 – Loader do sistema Windows

Fonte: Elaborada pelo autor (2023)

Para entra no nosso código basta executar o *run* ou apertar *F9* no teclado.

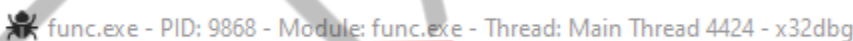


Figura 41 – EntryPoint do programa a ser debugado

Fonte: Elaborada pelo autor (2023)

Na posição 2 vemos o *disassembler* do nosso programa. Na posição 3 temos o dump de memória, já na posição 4 temos a *Stack* do programa. E na posição 5 os registradores na versão 32 bit.

A documentação dessa ferramenta é de extrema importância para você queira aprender mais sobre a tal. Abaixo podemos ver alguns simples comando que podemos utilizar para depurar nosso programa.

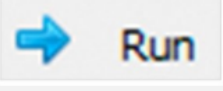
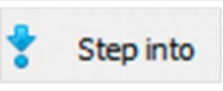
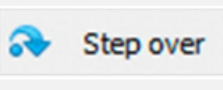
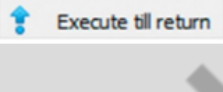
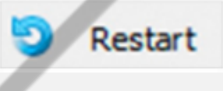
Comando	Atalho de Teclado	Descrição
 Run	F9	Executa o programa. Caso haja algum outro breakpoint no caminho, o <i>debugger</i> entrará no modo <i>paused</i>
 Step into	F7	Executa o programa <i>step-by-step</i> e caso encontre uma instrução <i>call</i> esse comando entrará dentro dessa função.
 Step over	F8	Executa o programa <i>step-by-step</i> , porém não entra dentro da função, apenas a executa.
 Execute till return	CTRL+F9	Executa uma função até sua instrução de retorno.
 Restart	CTRL+F2	Reinicia a execução do programa.

Tabela 9 – Comandos básicos do x64dbg
 Fonte: Elaborada pelo autor (2023)

Para mais informação a respeito de comando a serem utilizados no *debugger*, se refira a documentação.

5 DICAS EXTRAS

Como dito antes, aprender assembly exige tempo de dedicação para que você possa ficar familiarizado com as instruções e o contexto ao qual elas se encontram. Algumas dicas e sites que podem te ajudar nessa jornada são:

5.1 Godbolt & Dogbolt

Esse são dois sites que podem te ajudar a entender melhor a linguagem Assembly. Neles é possível visualizar como um código fonte ficará representando em assembly usando diferentes compiladores. Abaixo podemos ver um simples exemplo usando o Godbolt.

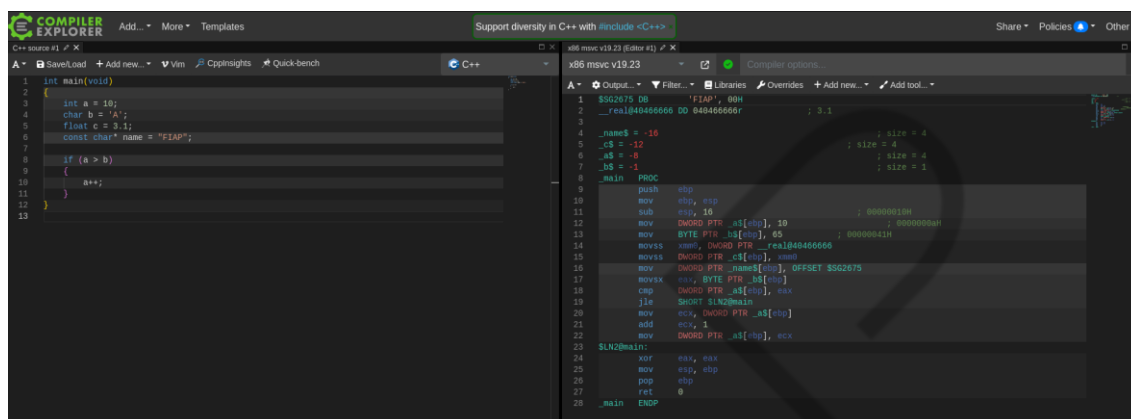


Figura 42 – Web disassembler Godbolt
Fonte: Elaborada pelo autor (2023)

Já o Dogbolt nos ajuda a visualizar a forma como diferentes disassembler interpretam o código compilado.

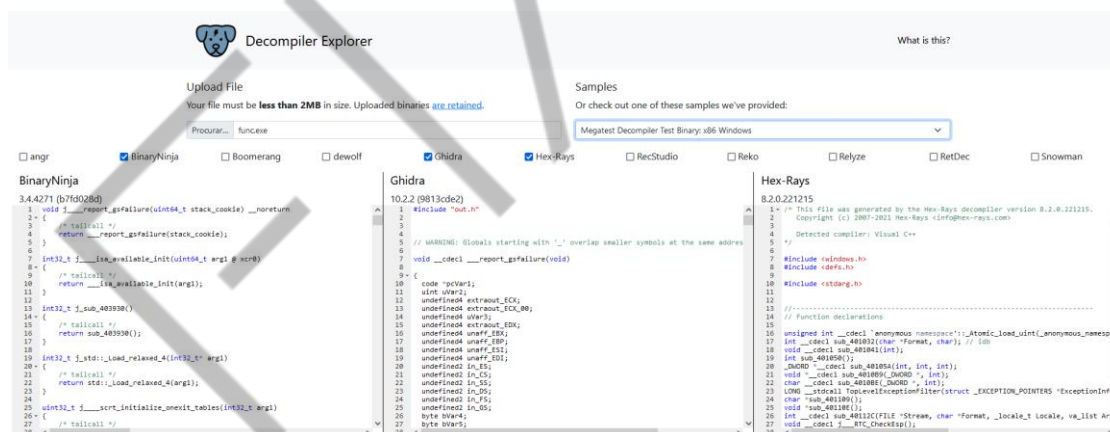


Figura 43 – Web disassembler Dogbolt
Fonte: Elaborada pelo autor (2023)

5.2 Livros e Dicas:

Alguns livros que podem ser de grande ajuda são:

- Reversing: Secrets of Reverse Engineering (por Eldad Eilam).

- Practical Malware Analysis: The hands-on guide to dissecting malicious software (por Michael Sikorski & Andrew Honing).

Esses são livros um pouco antigos, porém, as bases fornecidas por eles são de grande valor.

E a dica mais simples é simplesmente crie um código e compila e abre ele no IDA ou no Ghidra e tente interpretar.

Caso você goste de CTF (Capture The Flag), você pode encontrar diferentes desafios em sites com: crackmes.one.

CONCLUSÃO

Nesse capítulo você (muito provável) teve seu primeiro contato com a linguagem Assembly. Vimos que com esse conhecimento podemos interpretar outros softwares e nos ajudar na exploração de binários, análise de malware e até mesmo no desenvolvimento de softwares mais seguros e performáticos.

REFERÊNCIAS

ELDAD, Eilam. **Reversing**: Secrets of Reverse Engineering. Indianopolis: Wiley, 2005.

JAVAPOINT. LIFO Approach in data structure. s. d. Disponível em: <<https://www.javatpoint.com/lifo-approach-in-data-structure>>. Acesso em: 31 ago. 2023.

REACTOS. Gallery. s. d. Disponível em: <<https://reactos.org/gallery/>>. Acesso em: 31 ago. 2023.

SIKORSKI, Michael; HONING, Andrew. **Practical Malware Analysis**: The Hands-On Guide to Dissecting Malicious Software. San Francisco: No Starch Press, 2012.